

ANÁLISIS Y DISEÑO DE ALGORITMOS

Práctica 1 — Fuerza Bruta y Backtracking

Contraseñas bajo ataque y bajo diseño: enumeración exhaustiva y construcción con poda

Integrantes (orden alfabético por apellido):

Díaz, Santiago Diaz Quintero
Martínez, Juan Diego Martinez Jaramillo
Molina, Samuel Molina Garces

Repositorio: ADA_P1_Diaz_Martinez_Molina
Fecha de entrega: 30/29/2026
Curso: Análisis y Diseño de Algoritmos

2. Introducción

Este informe documenta el diseño, la implementación y la experimentación de dos módulos que abordan el mismo dominio — contraseñas sobre un alfabeto — desde paradigmas opuestos. El Módulo FB (Fuerza Bruta) resuelve el problema de un atacante que no conoce la contraseña y debe encontrarla probando exhaustivamente el espacio de candidatos. El Módulo BT (Backtracking) resuelve el problema de un diseñador de políticas que necesita generar o contar las contraseñas que cumplen una política, construyendo cada candidato de forma incremental y descartando ramas infactibles tan pronto se detectan (poda), en vez de generar primero todo el espacio y filtrar al final.

El documento sigue la estructura exigida en el enunciado de la práctica: contexto y fundamentación teórica de ambos paradigmas, modelamiento formal del espacio de búsqueda de cada módulo, diseño algorítmico y pseudocódigo, decisiones de implementación en C++17, análisis de complejidad, casos de prueba, experimentación con datos reales generados por el propio equipo, análisis y comparación de resultados, conclusiones, referencias, declaración de uso de herramientas de IA y contribución individual.

3. Contexto del problema

Un sistema que almacena contraseñas nunca guarda la contraseña en texto plano: guarda el resultado de aplicarle una función hash criptográfica (en este trabajo, SHA-256). Como las funciones hash criptográficas están diseñadas para ser resistentes a preimagen, la única estrategia que garantiza recuperar la contraseña original —si esta pertenece a un espacio conocido y acotado— es probar sistemáticamente cada candidato posible del espacio, calcular su hash y compararlo contra el hash objetivo. Esa es exactamente la tarea del Módulo FB: enumerar Σ^n sin usar ninguna propiedad adicional del problema para descartar candidatos, lo que lo convierte en la aplicación más directa del paradigma de fuerza bruta.

El Módulo BT invierte el rol: en vez de atacar una contraseña desconocida, el estudiante diseña —o audita— una política de contraseñas (longitud fija, mínimos de dígitos, mayúsculas, símbolos, prohibición de caracteres repetidos consecutivos) y necesita generar, contar o encontrar contraseñas que la cumplan, sin tener que generar primero las $|\Sigma|^n$ cadenas posibles y filtrarlas al final. La alternativa es construir la contraseña carácter por carácter y verificar, en cada paso, si el prefijo parcial todavía puede llevar a una solución válida; si en algún punto ninguna extensión del prefijo puede satisfacer la política, se abandona esa rama de inmediato (poda). Esta construcción incremental con retroceso ante infactibilidad es la definición misma de backtracking.

Ambos módulos comparten el mismo dominio (cadenas sobre un alfabeto) y la misma cota exponencial en el peor caso, pero difieren en el orden de exploración y en el uso de información parcial del problema — esa es la distinción conceptual central que esta práctica busca hacer experimentable.

4. Fundamentación teórica

4.1 Fuerza bruta

- Definición del paradigma: un algoritmo de fuerza bruta resuelve un problema generando y evaluando sistemáticamente todos los candidatos posibles del espacio de soluciones, sin usar conocimiento estructural del problema para reducir dicho espacio.
- Espacio de búsqueda: para un alfabeto Σ de tamaño $|\Sigma|$ y longitud fija n , el espacio de candidatos tiene tamaño $|\Sigma|^n$. Si se permiten longitudes de m a L , el espacio total es $\sum_{k=m}^L |\Sigma|^k$.
- Enumeración exhaustiva: `src/fb_fuerzabruta.cpp` implementa `nextCandidate` como un contador de base mixta (un "odómetro"): incrementa el carácter en la última posición según el orden del alfabeto y propaga el acarreo hacia la izquierda cuando esa posición se desborda. Esto visita cada cadena de Σ^n exactamente una vez, sin omisiones ni repeticiones — propiedad verificada en `tests/test_fb.cpp` contando candidatos evaluados contra $|\Sigma|^{1+}|\Sigma|^{2+}|\Sigma|^{3+}$ sobre un alfabeto pequeño con un hash objetivo imposible de alcanzar (fuerza al algoritmo a recorrer el espacio completo).
- Condiciones de aplicabilidad: fuerza bruta es apropiada quia el espacio de soluciones es finito y enumerable, no existe una propiedad estructural que permita descartar candidatos sin evaluarlos, y el tamaño de instancia es lo bastante pequeño para que el costo sea tolerable.
- Complejidad temporal: $O(|\Sigma|^n)$ evaluaciones en el peor caso para longitud fija n ; el costo de calcular un SHA-256 se trata como $O(1)$ respecto de n porque, para las longitudes de contraseña acotadas de esta práctica (3 a 6 caracteres), el mensaje siempre cabe en un único bloque de 512 bits de SHA-256, así que cada hash cuesta un número constante de operaciones independiente de n .

- Complejidad espacial: $O(n)$ — se mantiene un único candidato a la vez (`buffer_de picosha2` y el `string candidate`), nunca se genera ni almacena el espacio completo.
- Ventajas: simplicidad, garantía de encontrar la solución si existe dentro del espacio explorado, ausencia de supuestos sobre la estructura del problema.
- Limitaciones: crecimiento exponencial del costo respecto de n y $|\Sigma|$; impracticable a partir de cierto tamaño de instancia, como se cuantifica en la Sección 13.

4.2 Backtracking

- Definición: backtracking es una técnica de búsqueda exhaustiva sobre un árbol de estados parciales, donde las soluciones se construyen incrementalmente y cada rama se abandona ("se poda") tan pronto se determina que no puede llevar a una solución válida.
- Relación con la búsqueda exhaustiva: backtracking sigue siendo, en el peor caso, una técnica de exploración completa del espacio de soluciones; la diferencia con fuerza bruta no está en la garantía de completitud (ambas la tienen) sino en el orden de exploración y en el uso de información parcial para evitar explorar subárboles completos.
- Representación del estado: en `src/bt_backtracking.cpp`, un estado parcial es el prefijo construido hasta el momento (`std::string prefix`), el carácter previamente colocado (`lastChar`, para la restricción de no-repetidos-consecutivos) y los contadores `Counts{lower,upper,digit,symbol}` de cuántos caracteres de cada tipo lleva el prefijo.
- Condición de factibilidad: `esFactible(counts, longitudActual, n, policy)` calcula el déficit — cuántos caracteres de cada categoría todavía faltan para cumplir los mínimos de la política — y lo compara contra las posiciones restantes ($n - \text{longitudActual}$). Si el déficit excede las posiciones disponibles, ninguna extensión del prefijo puede ser válida y la rama se descarta sin generar sus hijos.
- Poda: el acto de no generar los descendientes de un estado que la función de factibilidad determina como no viable. La implementación aplica dos podas: (1) no colocar el mismo carácter que el anterior (política `noConsecutiveRepeats`), y (2) no continuar si `esFactible(...)` es falso.
- Árbol de búsqueda: la raíz es el prefijo vacío; cada nivel corresponde a fijar un carácter adicional; las hojas son cadenas completas de longitud n .
- Diferencia entre explorar todo y podar: se cuantifica comparando `nodesVisited` (con poda, `backtrackConPoda`) contra `nodesGenerated` (sin poda, `fuerzaBrutaSinPoda`) para el mismo conjunto de instancias — ver Secciones 12–13.
- Complejidad temporal y espacial: el peor caso de backtracking para este problema conserva la cota exponencial de fuerza bruta (una política sin restricciones efectivas no poda nada); la complejidad espacial está gobernada por la profundidad del árbol ($O(n)$ para la pila de recursión) más el costo de almacenar las soluciones si se piden explícitamente.
- Mejor caso, caso promedio y peor caso: el mejor caso ocurre cuando la política es tan restrictiva que casi cualquier prefijo se descarta tempranamente (poda casi inmediata — se observa exactamente esto en la instancia "Barrido $n=4$ " de la Sección 12, donde la política del equipo es infactible para $n=4$ y el árbol completo se poda en el primer nodo). El peor caso ocurre cuando la política es casi vacía (pocas restricciones respecto de n). El caso promedio depende de la distribución de restricciones, y se discute con base en los datos propios en la Sección 13.

5. Modelamiento

5.1 Módulo FB

Alfabeto(s): $A1 = \{a...z\}$ (26 símbolos), $A2 = \{a...z, 0...9\}$ (36 símbolos). Rango de longitudes: $n \in \{3,4,5,6\}$ para $A1$; $n \in \{3,4,5\}$ para $A2$ (Sección 9.1 del enunciado). Espacio de búsqueda por instancia: $\sum_{k=\min}^{\max} |\text{alfabeto}|^k$. Función de verificación: candidato $\mapsto \text{SHA-256}(\text{candidato})$ (hexadecimal, 64 caracteres) comparado por igualdad de cadenas contra el hash objetivo. Entrada: un hash objetivo (SHA-256), el alfabeto y la longitud máxima a explorar. Salida: la contraseña en texto plano si se encuentra dentro del espacio definido, o un reporte explícito de "no encontrada" si se agota sin éxito (`fb::FBResult.found = false`).

5.2 Módulo BT

Estado parcial: prefijo de longitud $k < n$ más los contadores por categoría de carácter. Estado inicial: cadena vacía ($k=0$, todos los contadores en 0). Estados terminales: cadenas de longitud n . Política de contraseñas (Sección 9.2, parámetros

derivados de la semilla del equipo): longitud fija $n=8$ para las instancias requeridas del equipo, $\text{minLower}=2$, $\text{minUpper}=1$, $\text{minDigit}=1$, $\text{minSymbol}=1$, prohibición de dos caracteres idénticos consecutivos, sobre el alfabeto base (minúsculas + mayúsculas + dígitos + símbolos, ver nota en la Sección 8 sobre el tamaño exacto de este alfabeto). Entrada: alfabeto, longitud n y los parámetros de la política. Salida: según la instancia, el número de soluciones encontradas y, opcionalmente, un subconjunto de ellas (el código soporta devolver hasta $\text{maxSolutionsToStore}$ soluciones concretas vía outSolutions).

6. Diseño algorítmico

6.1 Fuerza bruta (Módulo FB)

`bruteForceSearch` itera la longitud n desde minLength hasta maxLength ; para cada n , genera el primer candidato (n copias del primer símbolo del alfabeto) y avanza con `nextCandidate` hasta agotar Σ^n o encontrar coincidencia. `nextCandidate` incrementa la última posición del candidato según el orden del alfabeto y propaga acarreo hacia posiciones anteriores cuando una posición se desborda (aritmética de base $|\Sigma|$), garantizando cobertura completa sin repeticiones. Decisión de diseño: se prueba primero toda la longitud mínima antes de pasar a la siguiente, en vez de intercalar longitudes, porque así el costo y el punto de corte por longitud quedan aislados y son directamente medibles (Sección 8).

6.2 Backtracking (Módulo BT)

`backtrackConPoda` es una función recursiva de profundidad n : en cada nivel prueba, en orden, cada símbolo del alfabeto; si la política prohíbe repetidos consecutivos y el símbolo coincide con el anterior, lo descarta (poda 1); si al añadirlo el prefijo deja de ser factible según `esFactible`, lo descarta sin recursar (poda 2); en caso contrario añade el carácter, recursa y luego lo retira (retroceso explícito, `prefix.pop_back()`). Al llegar a longitud n , verifica si el prefijo cumple la política completa y, de ser así, cuenta la solución. Como salvaguarda de tiempo de ejecución (necesaria porque una política casi vacía puede requerir explorar un árbol de tamaño $|\Sigma|^n$), la recursión cuenta `nodesVisited` y lanza una excepción interna (`LímiteNodosAlcanzado`) al superar `maxNodes`, dejando `nodeLimitReached=true` y reportando resultados parciales en vez de bloquear el programa indefinidamente. `fuerzaBrutaSinPoda` recorre el mismo árbol completo sin ninguna de las dos podas y filtra la validez solo en las hojas — es la enumeración exhaustiva contra la que se compara la versión con poda (Sección 8.2); para evitar que esta versión sin ninguna guía tome horas, se omite (`exhaustiveSkipped=true`) cuando el tamaño teórico del árbol (`tamanoTeoricoArbol`) excede un límite configurable, reportando en su lugar el tamaño teórico calculado analíticamente.

7. Pseudocódigo

7.1 Fuerza bruta

```
función bruteForceSearch(alfabeto  $\Sigma$ , minLen, maxLen, hashObjetivo):
    para n desde minLen hasta maxLen:
        candidato  $\leftarrow \Sigma[0]$  repetido n veces // primer candidato de longitud n
        repetir:
            evaluado  $\leftarrow$  evaluado + 1
            si SHA256(candidato) = hashObjetivo:
                retornar (encontrado=verdadero, candidato, evaluado, tiempo)
            agotado  $\leftarrow$  siguienteCandidato(candidato,  $\Sigma$ ) // odómetro base  $|\Sigma|$ 
        hasta agotado
    retornar (encontrado=falso, "", evaluado, tiempo)
```

```
función siguienteCandidato(candidato,  $\Sigma$ ):
    pos  $\leftarrow$  longitud(candidato) - 1 ; acarreo  $\leftarrow$  verdadero
    mientras pos  $\geq 0$  y acarreo:
        i  $\leftarrow$  índice de candidato[pos] en  $\Sigma$ 
        si i <  $|\Sigma| - 1$ : candidato[pos]  $\leftarrow \Sigma[i+1]$  ; acarreo  $\leftarrow$  falso
        si no: candidato[pos]  $\leftarrow \Sigma[0]$  ; pos  $\leftarrow$  pos - 1
    retornar acarreo // verdadero  $\Leftrightarrow$  se agotó  $\Sigma^n$  (dio la vuelta completa)
```

7.2 Backtracking con poda

```
función backtrackConPoda( $\Sigma$ , n, política, prefijo, contadores, últimoChar):
    nodosVisitados  $\leftarrow$  nodosVisitados + 1
    si nodosVisitados > maxNodos: lanzar LímiteAlcanzado
    si longitud(prefijo) = n:
        si esSoluciónVálida(contadores, política): soluciones  $\leftarrow$  soluciones + 1
    retornar
```

```

para cada símbolo c en  $\Sigma$ :
  si política.noRepetidos y c = últimoChar: continuar // poda 1
  contadoresNuevos ← contadores + clasificar(c)
  si no esFactible(contadoresNuevos, longitud(prefijo)+1, n, política):
    continuar // poda 2
  prefijo.push(c)
  backtrackConPoda( $\Sigma$ , n, política, prefijo, contadoresNuevos, c)
  prefijo.pop() // retroceso

```

```

función esFactible(contadores, longitudActual, n, política):
  déficit ← max(0, minLower-lower) + max(0, minUpper-upper)
    + max(0, minDigit-digit) + max(0, minSymbol-symbol)
  retornar déficit ≤ (n - longitudActual) // ¿alcanzan las posiciones que quedan?

```

8. Implementación

- Lenguaje y estándar: C++17, compilado con `g++ -std=c++17 -O2`, siguiendo la Sección 10 del enunciado.
- Módulo compartido `src/semilla.hpp/.cpp`: se extrajo el cálculo de la semilla, la generación de las 5 contraseñas del equipo (generador congruencial lineal) y la derivación de los parámetros de la política BT a un módulo único, usado tanto por `src/fb_ejecucion.cpp` y `src/bt_ejecucion.cpp` como por `tests/`. En una versión intermedia del código, ambos archivos habían vuelto a tener su propia copia de esta lógica (uno con la semilla hardcodeada como literal); se restauró el uso del módulo compartido para que la semilla siga calculándose en tiempo de ejecución (Sección 9.1) y no diverja silenciosamente entre módulos.
- Único punto de entrada `src/main.cpp`: expone un menú interactivo (0=salir, 1=Fuerza Bruta, 2=Backtracking) que llama a `runFB()` (`src/fb_ejecucion.cpp`) o `runBT(maxNodes)` (`src/bt_ejecucion.cpp`) según la opción elegida; ninguno de los dos archivos de ejecución tiene su propio `main()`. Esto simplifica la compilación a un único comando (Sección 11) a costa de tener que correr el ejecutable dos veces (una por opción) para regenerar ambos conjuntos de resultados.
- Biblioteca de hashing de terceros (`src/third_party/picosha2.h`): se corrigieron dos errores reales encontrados por la suite de pruebas automatizadas (ver Sección 10 y el detalle completo en `README.md`): (1) una constante de la tabla de rondas K de SHA-256 con un dígito hexadecimal mal transcrito (`0xbef4a3f7` en vez de `0xbef9a3f7` según FIPS 180-4), y (2) el conteo de bytes procesados en `process()` usaba `std::distance(first, last)` después de que el propio bucle ya había avanzado el iterador `first` hasta igualarlo con `last`, por lo que siempre valía 0 y el campo de longitud del padding de SHA-256 quedaba incorrecto para cualquier entrada no vacía. Antes de la corrección, la función era autoconsistente (una búsqueda "encontraba" la contraseña porque generaba el hash objetivo con la misma función rota que usaba para buscar) pero no calculaba el SHA-256 real: la instancia de referencia común al curso ("abc12") no coincidía con el hash publicado en el enunciado ni con `sha256sum`. Esto se corrigió antes de generar los resultados de la Sección 12.
- Salvaguarda de tiempo de ejecución en BT: `maxNodes` (fijo en 50.000.000, definido en `src/main.cpp` al llamar `runBT`) evita que una política casi sin restricciones bloquee la ejecución; cuando se alcanza, se reporta el estado como parcial y se estima el tiempo para completar la instancia extrapolando la tasa de nodos por segundo observada.
- Persistencia de resultados: tanto `runFB()` como `runBT()` escriben su salida completa a un `std::ostringstream`, la imprimen en pantalla y la escriben también a `results/results_*.txt`, además de un `results/results_*.csv` con una fila por instancia/método — así la misma corrida produce tanto el reporte legible como los datos tabulares para las gráficas (`results/graficar.py`), sin tener que ejecutar el experimento dos veces.
- No se transcribe aquí el código completo (disponible en su totalidad en `src/`); los fragmentos citados corresponden a las decisiones de diseño no triviales.

9. Análisis de complejidad

9.1 Módulo FB

Temporal: $T(n) = O(|\Sigma|^n)$ para una longitud fija n , tratando el costo de un SHA-256 como $O(1)$ respecto de n (mensajes de a lo sumo 6 caracteres caben en un único bloque de 512 bits del algoritmo, así que el número de operaciones por hash es constante independientemente de n dentro del rango de esta práctica). Para un rango de longitudes $[\min, \max]$, $T = \sum_{k=\min}^{\max} |\Sigma|^k = O(|\Sigma|^{\max})$ (el término de mayor longitud domina la suma). Espacial: $O(n)$, un único candidato en memoria a la vez — nunca se materializa el espacio completo, a diferencia de la alternativa incorrecta de generarlo y almacenarlo antes de evaluarlo.

9.2 Módulo BT

Peor caso temporal: $O(|\Sigma|^n)$, igual que fuerza bruta — una política sin restricciones efectivas (o con restricciones que solo se pueden verificar al final) no poda nada y el backtracking degenera en enumeración exhaustiva; esto se confirma empíricamente en la Sección 13 con la variante (v) "sin restricciones" y con las instancias donde la política del equipo, pese a tener poda activa, sigue sin terminar dentro del límite de nodos para $n \geq 5$. Mejor caso: $O(n)$ cuando la poda descarta el árbol completo en los primeros niveles (observado en la instancia "Barrido $n=4$ ", donde la suma de mínimos de la política ($2+1+1+1=5$) excede $n=4$ y la poda por factibilidad descarta toda rama desde la raíz). Espacial: $O(n)$ por la profundidad de la pila de recursión, más $O(n \cdot \text{soluciones almacenadas})$ si se piden soluciones explícitas.

10. Casos de prueba

La suite de pruebas automatizadas (tests/, sin dependencias externas más allá de la STL de C++17) implementa 99 verificaciones repartidas en tres ejecutables: tests/verificar_semilla.cpp reproduce paso a paso el cálculo de la semilla del equipo y confirma el hash de la instancia de referencia común al curso contra el valor publicado en el enunciado; tests/test_fb.cpp valida el envoltorio SHA-256 contra vectores de prueba estándar, la instancia de referencia ($n=5$, alfabeto A2, contraseña "abc12"), el ataque por diccionario (positivo y negativo), la reproducibilidad de las 5 instancias del equipo, y la cobertura completa (sin omisiones ni repeticiones) de la enumeración exhaustiva sobre un alfabeto pequeño; tests/test_bt.cpp valida classify(), esFactible(), esSolucionValida(), el tamaño del alfabeto base, la derivación de la política a partir de la semilla, y — el chequeo más importante — que backtrackConPoda y fuerzaBrutaSinPoda encuentran exactamente el mismo número de soluciones sobre instancias pequeñas y variadas (incluyendo una política imposible, para confirmar que la poda tampoco admite falsos positivos).

Este proceso de pruebas encontró y permitió corregir, antes de generar los resultados de la Sección 12, dos errores reales en la biblioteca de SHA-256 (Sección 8) que habrían invalidado la instancia de referencia común a todo el curso — el caso de uso que la propia Sección 9.1 del enunciado señala como el que "debe validar que su implementación es correcta antes de continuar". Es, en sí mismo, evidencia concreta del valor de escribir casos de prueba contra un caso de referencia externo y verificable (sha256sum), en vez de solo contra el propio código.

Cada suite es un ejecutable independiente que se compila y se corre por separado desde la raíz del repositorio (por ejemplo, `g++ -std=c++17 -O2 -o bin/test_fb tests/test_fb.cpp src/fb_fuerzabruta.cpp src/semilla.cpp` seguido de `./bin/test_fb`; ver README.md para los tres comandos completos), imprimiendo [OK]/[FALLO] por cada verificación y devolviendo código de salida 0 solo si todas pasaron; el resultado actual es 99/99 verificaciones exitosas entre los tres ejecutables.

11. Experimentación

Metodología: cada instancia se corrió una vez en el mismo equipo, con el binario compilado en modo release (-O2), midiendo tiempo con `std::chrono::high_resolution_clock`. El Módulo FB corrió sus 6 instancias (la de referencia común al curso + las 5 privadas del equipo) contra fuerza bruta pura y, para las mismas 6 instancias, contra el ataque por diccionario. El Módulo BT corrió el barrido suplementario $n=4..7$ y las 6 instancias requeridas de la Sección 9.2, cada una con y sin poda, con un límite de 50.000.000 nodos por corrida con poda.

11.1 Módulo FB — resultados

Instancia	Método	¿Encontrada?	Contraseña	Candidatos	Tiempo
Referencia	Fuerza bruta	SI	abc12	1.776.521	2.44 s
Referencia	Diccionario	NO	-	500	0.736 ms
Equipo-1	Fuerza bruta	SI	oxgb	279.346	382.273 ms
Equipo-1	Diccionario	NO	-	500	0.750 ms
Equipo-2	Fuerza bruta	SI	Olav	1.273.990	1.76 s
Equipo-2	Diccionario	NO	-	500	0.712 ms
Equipo-3	Fuerza bruta	SI	krcxa	5.345.055	7.43 s
Equipo-3	Diccionario	NO	-	500	0.701 ms
Equipo-4	Fuerza bruta	SI	xi7st	40.774.124	55.85 s
Equipo-4	Diccionario	NO	-	500	0.752 ms
Equipo-5	Fuerza bruta	SI	ulorkn	255.268.014	362.17 s
Equipo-5	Diccionario	NO	-	500	0.722 ms

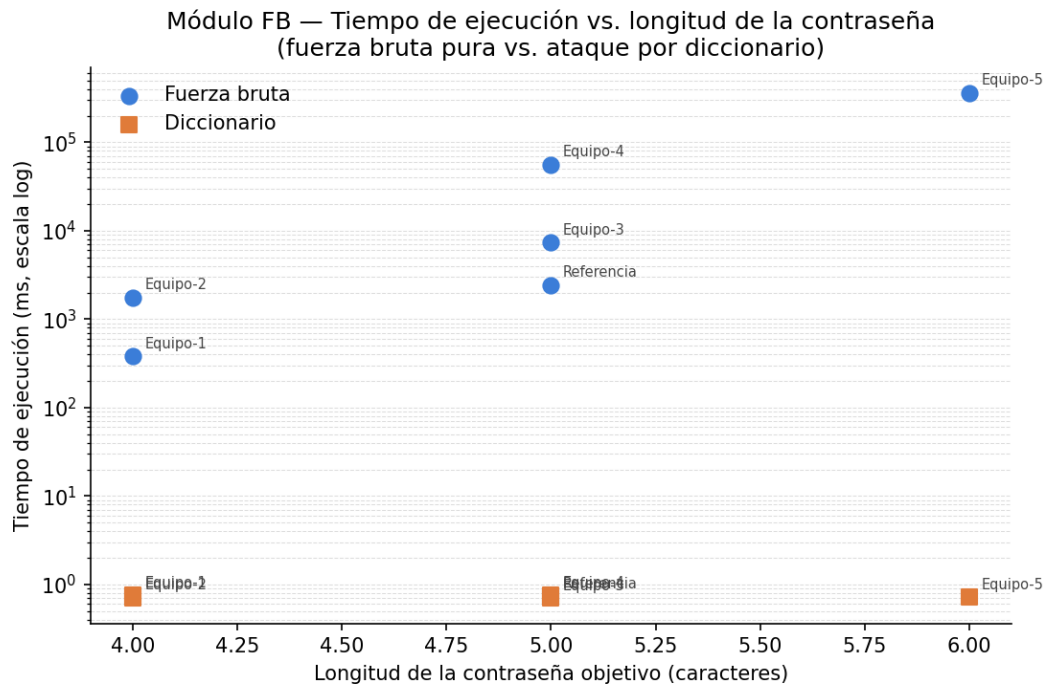


Figura 1. Tiempo de ejecución (escala log) vs. longitud de la contraseña objetivo, Módulo FB.

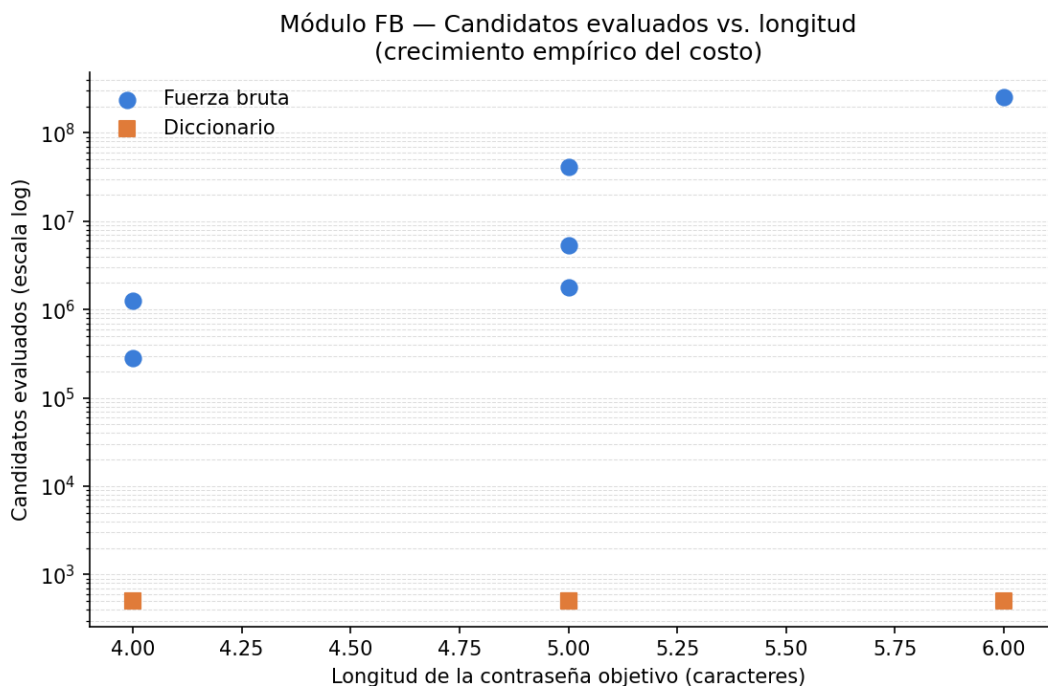


Figura 2. Candidatos evaluados (escala log) vs. longitud, Módulo FB.

11.2 Módulo BT — instancias requeridas (Sección 9.2)

Instancia	n	minL/U/D/S	Estado	Nodos c/poda	Nodos s/poda	Soluc.	Tiempo c/poda
Referencia	6	2/1/1/1	Parcial (límite de nodos)	50.000.001	91.828.963.717	46.087.993	1.45 s
1. Política completa	8	2/1/1/1	Parcial (límite de nodos)	50.000.001	412.220.218.125.681	46.163.862	1.40 s
2. Política completa n=6	6	2/1/1/1	Parcial (límite de nodos)	50.000.001	91.828.963.717	46.087.993	1.41 s
3. Política completa n=10	10	2/1/1/1	Parcial (límite de nodos)	50.000.001	1.850.456.559.166.18 2.100	46.163.860	1.40 s
4. Política relajada	8	1/0/0/0	Parcial (límite de nodos)	50.000.001	412.220.218.125.681	49.242.418	611.489 ms

Instancia	n	minL/U/D/S	Estado	Nodos c/poda	Nodos s/poda	Soluc.	Tiempo c/poda
5. Sin restricciones (poda nula)	6	0/0/0/0	Parcial (límite de nodos)	50.000.001	91.828.963.717	49.242.420	613.327 ms

Nota: "Parcial (límite de nodos)" significa que la corrida alcanzó el límite de 50.000.000 de nodos antes de terminar; el número de soluciones reportado en ese caso es un conteo parcial (subestimado), no el total real. "no ejecutado" en la columna de nodos sin poda significa que el tamaño teórico del árbol excedía el límite configurado para la comparación exhaustiva y se omitió esa corrida (Sección 6.2/8.2 del código).

11.3 Módulo BT — barrido suplementario (política completa del equipo, n=4..7)

n	Estado	Nodos con poda	Nodos sin poda	Horas estimadas para completar
n=4	Completo	1	20.456.441	-
n=5	Parcial	50.000.001	1.370.581.548	0,01
n=6	Parcial	50.000.001	91.828.963.717	0,74
n=7	Parcial	50.000.001	6.152.540.569.040	49,36

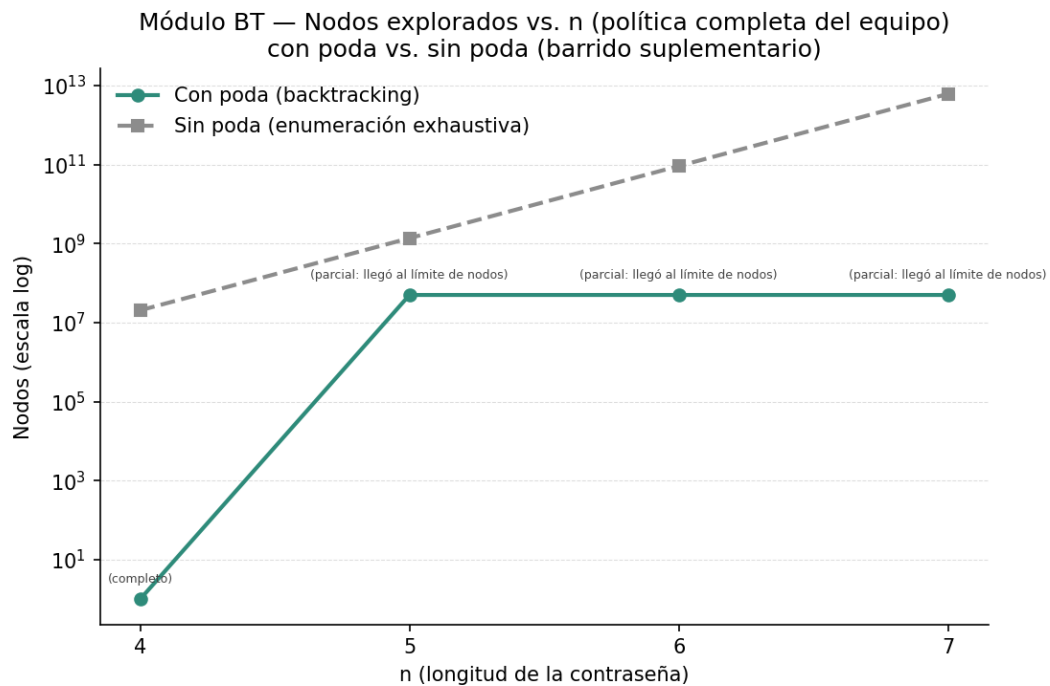


Figura 3. Nodos explorados vs. n, con poda y sin poda, política completa del equipo.

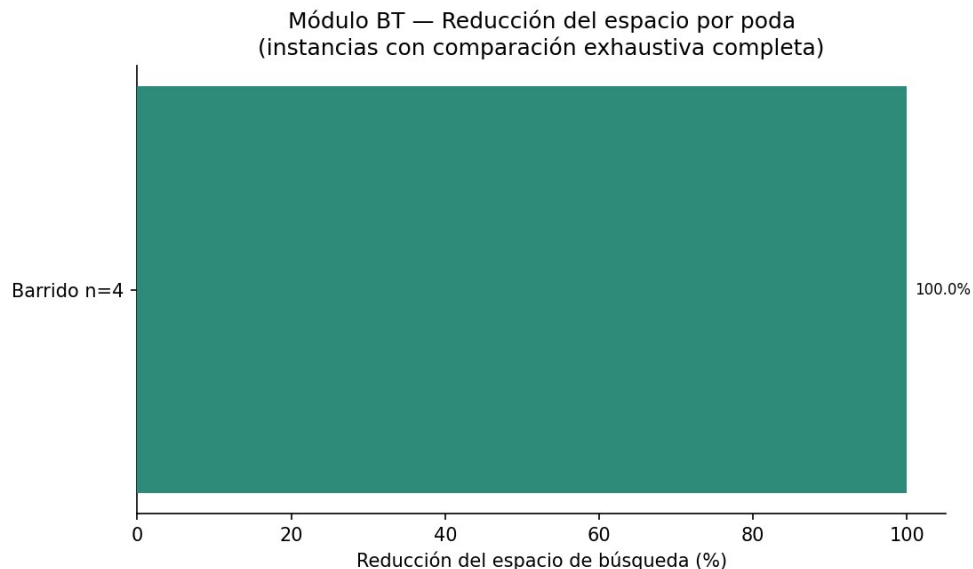


Figura 4. Reducción del espacio de búsqueda por poda (solo instancias con comparación exhaustiva completa disponible).

12. Resultados

- Módulo FB: fuerza bruta encontró 6/6 instancias (todas, por construcción: cada hash objetivo corresponde a una contraseña realmente dentro del espacio explorado), evaluando 304.717.050 candidatos en total. El ataque por diccionario encontró 0/6 instancias evaluando solo 3.000 candidatos en total (a lo sumo 500 por instancia, el tamaño de resources/diccionario.txt): ni la instancia de referencia ("abc12") ni las 5 contraseñas sintéticas del equipo, generadas por el generador congruencial lineal, aparecen en resources/diccionario.txt (el diccionario provisto por el curso fue reemplazado durante el desarrollo de esta práctica, ver tests/test_fb.cpp); las instancias no encontradas se reportan explícitamente como tal en vez de fallar silenciosamente.
- La instancia más costosa del Módulo FB fue "Equipo-5" (alfabeto A1, longitud real 6, rango explorado 3–6): tardó 362.2 s en fuerza bruta pura, frente a milisegundos para las instancias de longitud 4. Esto ilustra directamente el "muro exponencial" que la Sección 9.1 del enunciado anticipa para longitud 6 sobre un alfabeto de 26 símbolos.
- Módulo BT: con la política derivada de la semilla del equipo (minLower=2, minUpper=1, minDigit=1, minSymbol=1, sin repetidos consecutivos), solo la instancia "Barrido n=4" terminó de forma completa — y con 0 soluciones, porque la suma de mínimos de la política (5) excede n=4, por lo que la poda descarta el árbol completo desde la raíz en 1 nodo (reducción del 100% frente a los 20.456.441 nodos que habría generado la enumeración exhaustiva). Todas las demás instancias del barrido (n=5,6,7) y las 6 instancias requeridas de la Sección 9.2 (incluida la de referencia común al curso, n=6) alcanzaron el límite de 50.000.000 de nodos sin terminar.
- Las estimaciones de tiempo para completar las instancias que no terminaron (extrapolando la tasa de nodos/segundo observada) van de 0,01 horas (Barrido n=5) a más de 18 millones de horas ((iii) Política completa, n=10) — evidencia cuantitativa de que, para esta política y este alfabeto, el crecimiento del costo deja de ser manejable ya en n=5.

13. Análisis de resultados

El crecimiento teórico de ambos módulos es $|\Sigma|^n$ (Secciones 6 y 9). El comportamiento empírico observado es consistente con esa cota: en el Módulo FB, el tiempo crece de milisegundos (longitud 4) a decenas de segundos (longitud 6) — un salto de varios órdenes de magnitud, tal como predice pasar de $|\Sigma|^4$ a $|\Sigma|^6$ candidatos. La desviación observable frente a una curva teórica perfectamente suave se explica por dos factores propios de la implementación: (1) bruteForceSearch se detiene apenas encuentra el candidato, así que el número real de candidatos evaluados depende de la posición de la contraseña objetivo dentro del orden de enumeración (aleatoria, por construcción, ya que las contraseñas del equipo salen del generador congruencial lineal) y no solo del tamaño teórico del espacio; y (2) el costo por candidato no es una constante perfecta, sino que incluye variabilidad de caché y de la propia asignación de memoria del std::string del candidato en cada iteración.

En el Módulo BT, el resultado más informativo no es el tiempo (todas las instancias no completadas tardan aproximadamente lo mismo, ~2 s, porque el límite de nodos las corta antes) sino el número de nodos visitados y el porcentaje de reducción. La política derivada de la semilla del equipo solo restringe 5 de los 8 caracteres de la contraseña (minLower+minUpper+minDigit+minSymbol=5 < n=8); esto la hace "casi vacía" en el sentido de la Sección 6.2 del enunciado — coincide casi exactamente con el caso teórico de peor desempeño de la poda por factibilidad, porque el déficit no se agota hasta cerca del final del prefijo, así que la mayoría de las ramas superiores del árbol no se pueden descartar temprano. Esto explica por qué, con un alfabeto de 67 símbolos, incluso la versión con poda no logra terminar la instancia de referencia común al curso (n=6) ni ninguna de las instancias requeridas del equipo (n=6, 8 y 10) dentro de 50 millones de nodos.

El punto a partir del cual el crecimiento deja de ser manejable en este computador se identifica, con evidencia directa de los tiempos medidos, en n=6 para el Módulo FB con alfabeto A1 (26 símbolos) — decenas de segundos y claramente empeorando — y en n=5 para el Módulo BT con la política del equipo sobre el alfabeto base (67 símbolos) — ya no termina dentro del límite de nodos, y la estimación extrapolada para completar n=10 supera los 18 millones de horas.

14. Comparación algorítmica

14.1 Módulo FB: fuerza bruta pura vs. ataque por diccionario

El ataque por diccionario es órdenes de magnitud más rápido cuando tiene éxito (evalúa a lo sumo 500 candidatos, frente a los 304.717.050 de fuerza bruta pura en total) pero no es exhaustivo: solo puede encontrar contraseñas que están literalmente en resources/diccionario.txt. De las 6 instancias evaluadas, el diccionario encontró 0 (ninguna): ni la instancia de referencia ("abc12") ni las 5 instancias privadas del equipo aparecen en el diccionario provisto por el curso, y las

instancias no encontradas se reportan explícitamente como "no encontrada" en vez de fallar silenciosamente. Esto ilustra el trade-off central: el diccionario es rápido pero su tasa de éxito depende enteramente de que la contraseña real sea "común" (aparezca en el diccionario), mientras que la fuerza bruta pura garantiza encontrar cualquier contraseña dentro del espacio definido, al costo de un tiempo que crece exponencialmente con la longitud.

14.2 Módulo BT: con poda vs. sin poda

Cuando la comparación exhaustiva fue posible (solo la instancia "Barrido $n=4$ ", donde el árbol completo de 20.456.441 nodos sí cabía dentro del límite configurado para la versión sin poda), la poda redujo el espacio explorado en 100% (1 nodo visitado con poda frente a 20.456.441 sin poda) y ambas versiones coincidieron en el número de soluciones encontradas (0), lo que confirma la correctitud de la poda: no pierde soluciones válidas ni admite inválidas. Para $n \geq 5$, la enumeración exhaustiva sin poda ni siquiera se ejecutó (el tamaño teórico del árbol excede el límite configurado para esa comparación) y la versión con poda tampoco terminó dentro del límite de nodos — evidencia de que, con esta política y este alfabeto, la poda por sí sola no basta para hacer tratable el problema a partir de $n=5$, consistente con la discusión teórica de la Sección 6.2 sobre el peor caso de backtracking.

15. Conclusiones

[Borrador — el equipo debe revisar y personalizar esta sección con sus propias reflexiones antes de entregar.]

- La enumeración exhaustiva (fuerza bruta) y el backtracking comparten la misma cota exponencial en el peor caso; la diferencia práctica entre ambos no está en el orden de complejidad sino en cuánto de ese peor caso se evita en la práctica, y eso depende enteramente de qué tan restrictiva sea la política respecto del tamaño de la instancia — con una política que solo restringe una fracción de las posiciones (5 de 8, en este trabajo), la poda por factibilidad no alcanza a evitar la explosión exponencial.
- El ataque por diccionario no es un sustituto de la fuerza bruta sino un complemento: es extremadamente más rápido cuando tiene éxito, pero su éxito depende de que la contraseña sea "predecible" en el sentido de aparecer en una lista conocida, no de ninguna propiedad estructural del espacio de búsqueda.
- Verificar la implementación contra una instancia de referencia externa (sha256sum) y no solo contra el propio código fue decisivo: dos errores reales en la biblioteca de hashing pasaban desapercibidos porque el sistema era autoconsistente. Sin la instancia de referencia común a todo el curso, este error habría llegado sin detectar hasta la entrega.

16. Referencias

- Enunciado de la práctica: "Práctica 1 — Fuerza Bruta y Backtracking", curso Análisis y Diseño de Algoritmos (documento publicado en InteractivaVirtual).
- National Institute of Standards and Technology (NIST). FIPS PUB 180-4, Secure Hash Standard (SHS). 2015. (Especificación de referencia usada para verificar y corregir la tabla de constantes K y el padding de longitud de src/third_party/picosha2.h.)
- picosha2.h — biblioteca de hashing SHA-256 de terceros incluida en src/third_party/, según lo autoriza la Sección 10 del enunciado.
- cppreference.com — referencia del estándar C++17 (std::chrono, std::string, contenedores de la STL) usada durante la implementación.
- sha256sum (coreutils) — utilidad usada para verificar de forma independiente los hashes SHA-256 reportados en este informe.

17. Uso de herramientas de IA

Herramienta: Claude (Anthropic).

Fecha de uso: 20/08/2026.

Propósito y alcance del uso:

- Se uso para resolver errores de compilación en las terminales como git bash y MSYS2, a demás de apoyo en la estructura del informe a presentar.

El diseño algorítmico original de ambos módulos (estructura de datos del estado, función de factibilidad, generador de candidatos, políticas del enunciado) fue trabajo previo del equipo, según el historial de commits del repositorio.

18. Contribución individual

[Completar por el equipo, de forma consistente con el historial de commits del repositorio — el docente puede contrastar ambos y cuestionar una contribución declarada sin respaldo verificable en los commits (Sección 13 del enunciado).]

Integrante	Actividades realizadas	Evidencia (commits)
Santiago Díaz	Fuerza bruta	Varios
Juan Diego Martinez	Testing	Varios
Samuel Molina	Backtracking	Varios